

[Name der Hochschule]

[Art der Arbeit]

[Titel der Fallstudie zum Technologie-Template]

[Untertitel]

[Aufgabenstellung]

Kurs: [Kurs / Modul]
Studiengang: [Studiengang]
Autor: [Vorname Nachname]
Matrikelnummer: [Matrikelnummer]
Tutor: [Tutor / Betreuer]
Abgabedatum: [Monat Jahr]

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	1
1.4 Fragestellungen	1
1.5 Methodisches Vorgehen	1
1.6 Aufbau der Fallstudie	2
2 Anforderungen an das Technologie-Template	2
2.1 Zielbild des Technologie-Templates	2
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	2
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	2
2.4 Qualitäts- und Wartbarkeitsanforderungen	2
2.5 Abgrenzung	2
3 Architektur des Technologie-Templates	2
3.1 Gesamtarchitektur	2
3.2 Frontend mit Vite und TypeScript	3
3.3 Backend mit FastAPI	4
3.4 Desktop-Integration mit Tauri und Rust	4
3.5 Shared Contracts und Schnittstellen	4
3.6 Konfigurationsmodell	4
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	5
4 Projektprofile und Feature-Modell	5
4.1 Grundprinzip des Profilmodells	5
4.2 Profil Web-only	7
4.3 Profil Web-cloud	7

4.4	Profil Desktop-local	7
4.5	Profil Desktop-cloud	7
4.6	Profil Full-platform	7
4.7	Optionale Capabilities	7
4.8	Single-Repository-Strategie	7
5	Tooling und Entwicklungsworkflow	7
5.1	Rolle von control.py	7
5.2	Projektinitialisierung und Generierung	8
5.3	Systemprüfung und Installation	8
5.4	Entwicklungsbetrieb	8
5.5	Tests und Builds	9
5.6	Release- und Packaging-Workflow	9
5.7	Entwickler-Onboarding	9
6	Qualitätssicherung und Verifikation	9
6.1	Teststrategie	9
6.2	Continuous Integration	10
6.3	Abnahme und technische Verifikation	10
6.4	Reproduzierbarkeit	10
6.5	Security und Konfigurationsgrenzen	10
6.6	Extern verbleibende Prüfungen	10
7	Bewertung des Technologie-Templates	10
7.1	Produktivitätswirkung	10
7.2	Stärken	10
7.3	Schwächen und Grenzen	11
7.4	Wartungsaufwand	11
7.5	Vergleich mit alternativen Template-Strategien	11
7.6	Gesamtbewertung	11
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	11
8.1	Geeignete Projekttypen	11
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	11
8.3	Analyse von Kundenanforderungen	11

8.4	Auswahl des Projektprofils	11
8.5	Generierung eines Kundenprojekts	11
8.6	Überführung in die Produktentwicklung	12
9	Schlussbetrachtung	12
9.1	Zusammenfassung der Ergebnisse	12
9.2	Beantwortung der Fragestellungen	12
9.3	Kritische Würdigung	12
9.4	Ausblick	12
9.5	Fazit	12
	Literaturverzeichnis	13

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	1
2	Gesamtarchitektur des Technologie-Templates	2
3	Komponenten und Verantwortungsgrenzen im Master Repository	2
4	Profilabhängige Laufzeitarchitektur	3
5	Konfigurationspriorität und Sicherheitsgrenzen	4
6	Profilmodell auf Basis eines gemeinsamen Master-Templates	5
7	Strukturelle Feature-Zuordnung der Projektprofile	5
8	Entscheidungsstruktur für die Auswahl eines Projektprofils	6
9	Befehlsgruppen des zentralen Projekt-Toolings	7
10	Workflow der profilgesteuerten Projektgenerierung	8
11	Grundstruktur des Entwicklungsworkflows	8
12	Providerneutrale Deployment-Architektur	9
13	Plattformübergreifendes Modell für Desktop-Releases	9
14	Nachweiskette der technischen Verifikation	9
15	Struktur der profilbezogenen Verifikationsmatrix	10
16	Offene Bewertungsstruktur für die Projekteignung	11
17	Prozess vom Kundenbedarf bis zur Auslieferung	11

Tabellenverzeichnis

1	Struktur zur Dokumentation des Technologie-Stacks	3
2	Struktur zur Dokumentation der Projektprofile	6
3	Struktur zur Dokumentation der technischen Verifikation	10
4	Struktur zur Gegenüberstellung von Stärken und Grenzen	10
5	Struktur zur Bewertung der Projekteignung	11

Abkürzungsverzeichnis

Abk.	Bedeutung
------	-----------

1 Einleitung

1.1 Ausgangslage

1.2 Problemstellung

1.3 Zielsetzung

1.4 Fragestellungen

1.5 Methodisches Vorgehen



Abbildung 1: Methodische Schritte der Fallstudie

1.6 Aufbau der Fallstudie

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

2.4 Qualitäts- und Wartbarkeitsanforderungen

2.5 Abgrenzung

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

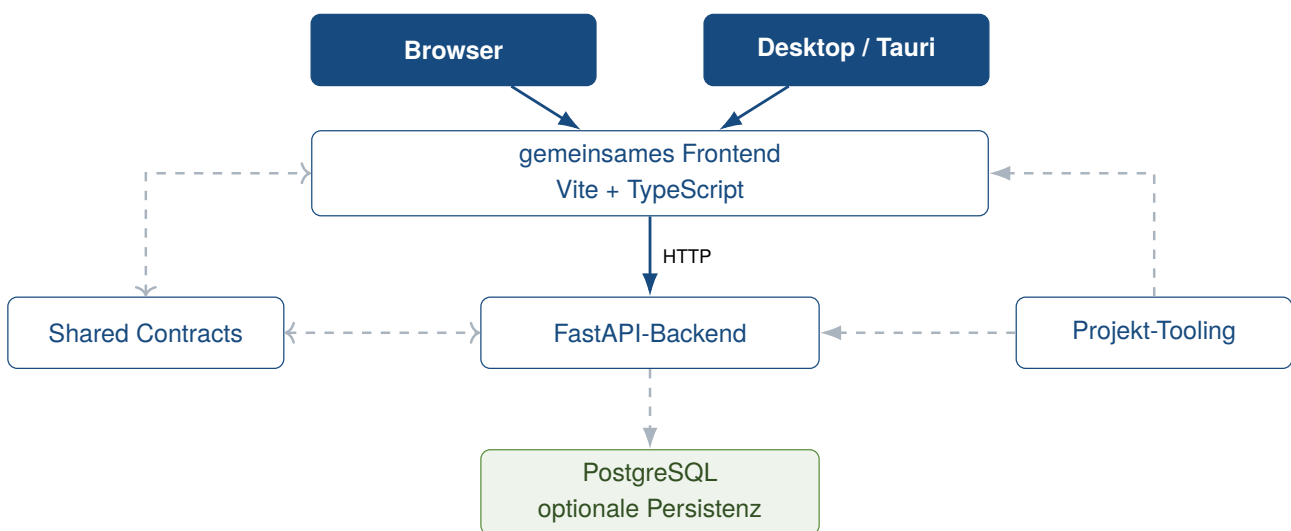


Abbildung 2: Gesamtarchitektur des Technologie-Templates

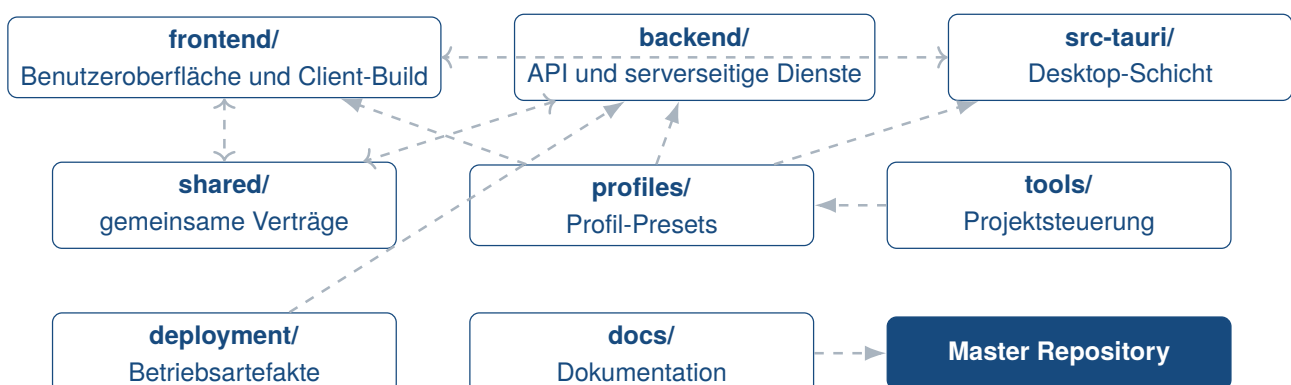
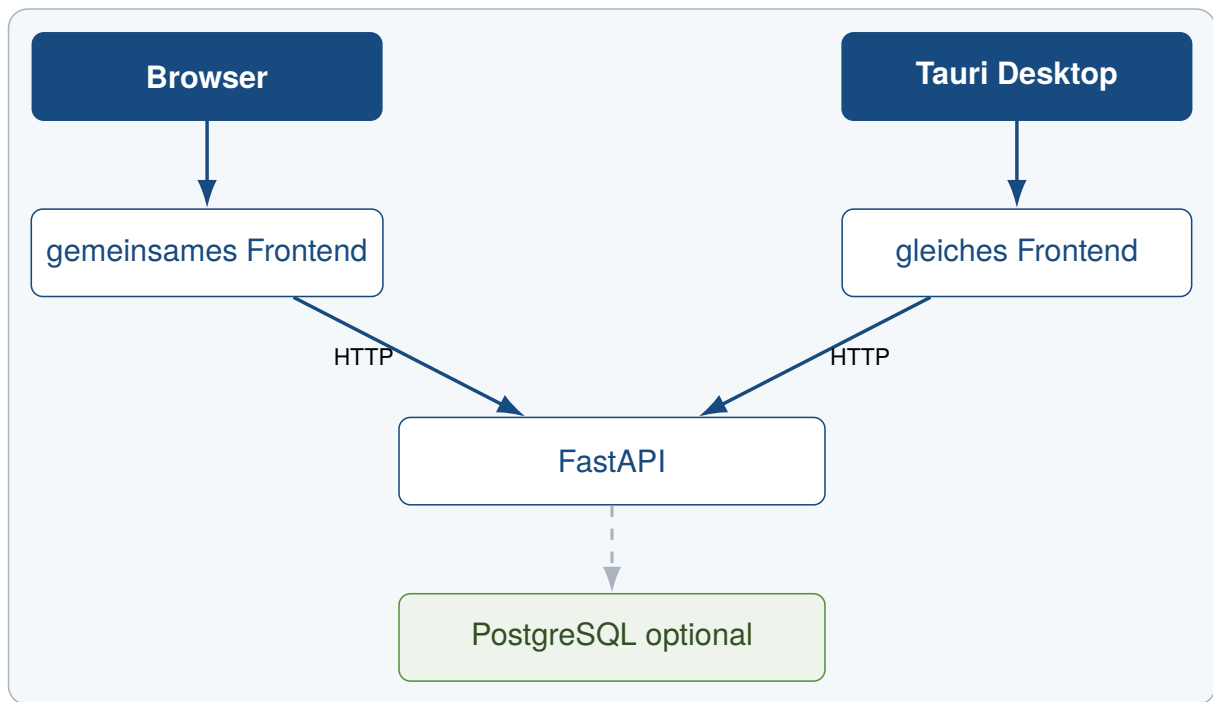


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository



Komponentenbestand abhängig vom gewählten Profil

Abbildung 4: Profilabhängige Laufzeitarchitektur

3.2 Frontend mit Vite und TypeScript

Tabelle 1: Struktur zur Dokumentation des Technologie-Stacks

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
-------------	-------------------	------------------	--------------

3.3 Backend mit FastAPI

3.4 Desktop-Integration mit Tauri und Rust

3.5 Shared Contracts und Schnittstellen

3.6 Konfigurationsmodell

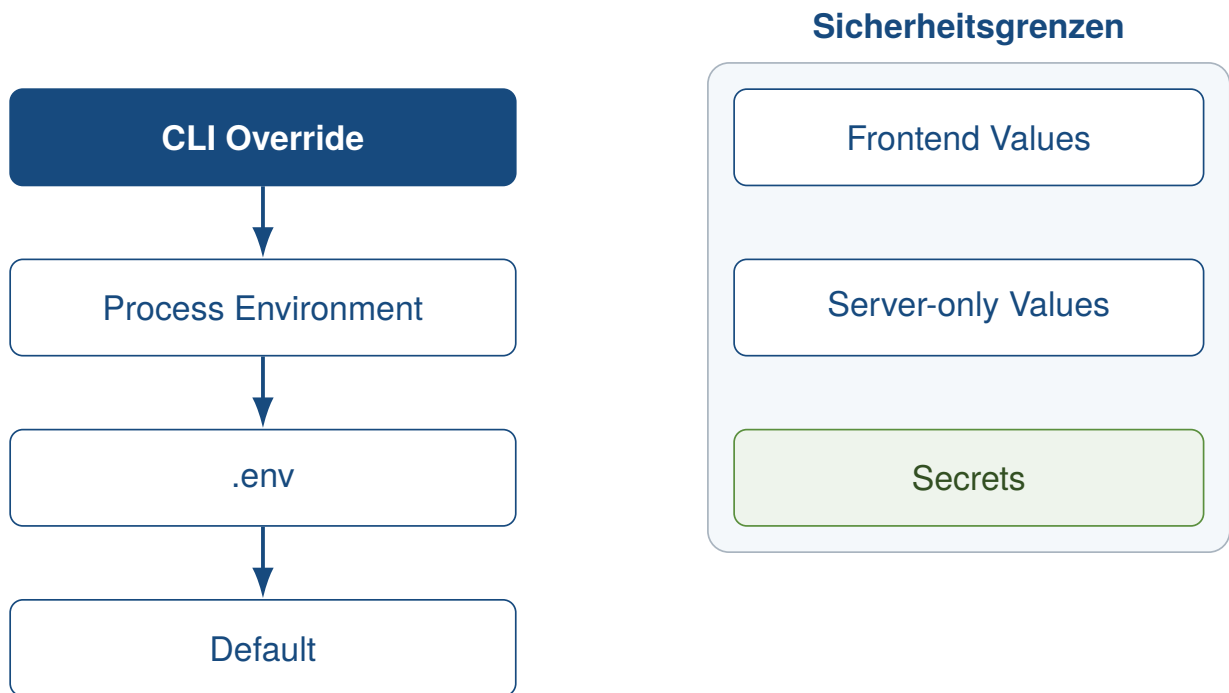


Abbildung 5: Konfigurationspriorität und Sicherheitsgrenzen

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

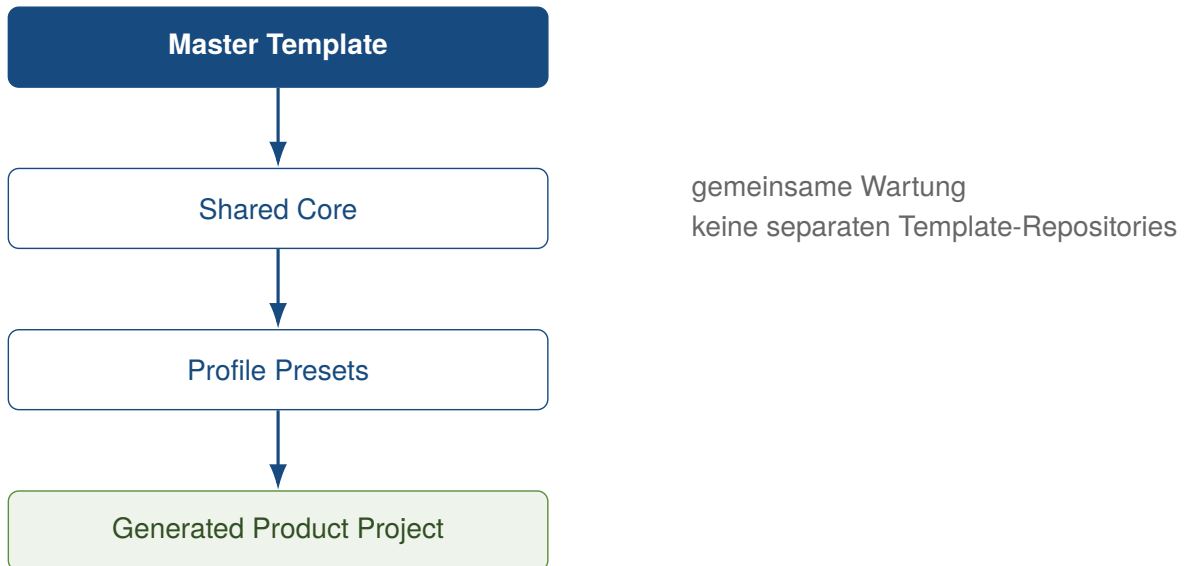


Abbildung 6: Profilmodell auf Basis eines gemeinsamen Master-Templates

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	–
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	–
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 7: Strukturelle Feature-Zuordnung der Projektprofile

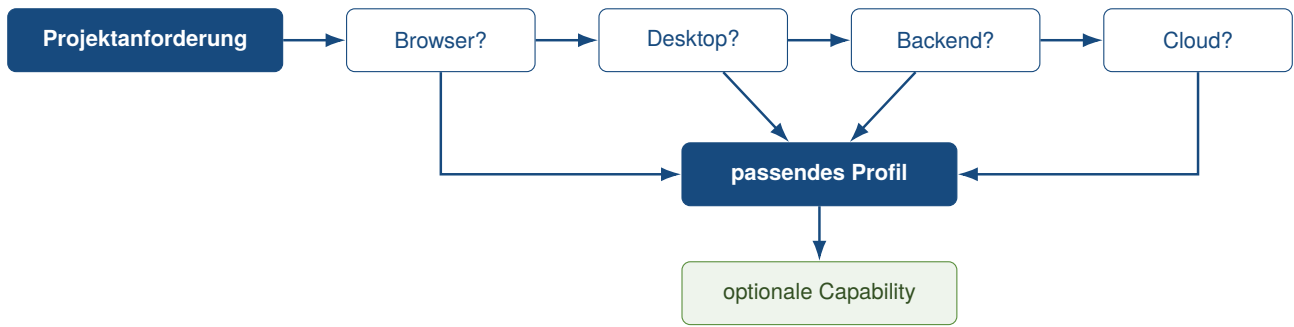


Abbildung 8: Entscheidungsstruktur für die Auswahl eines Projektprofils

Tabelle 2: Struktur zur Dokumentation der Projektprofile

Profil	Plattformbezug	enthaltene Komponenten	optionale Capabilities
--------	----------------	------------------------	------------------------

4.2 Profil Web-only

4.3 Profil Web-cloud

4.4 Profil Desktop-local

4.5 Profil Desktop-cloud

4.6 Profil Full-platform

4.7 Optionale Capabilities

4.8 Single-Repository-Strategie

5 Tooling und Entwicklungsworkflow

5.1 Rolle von control.py

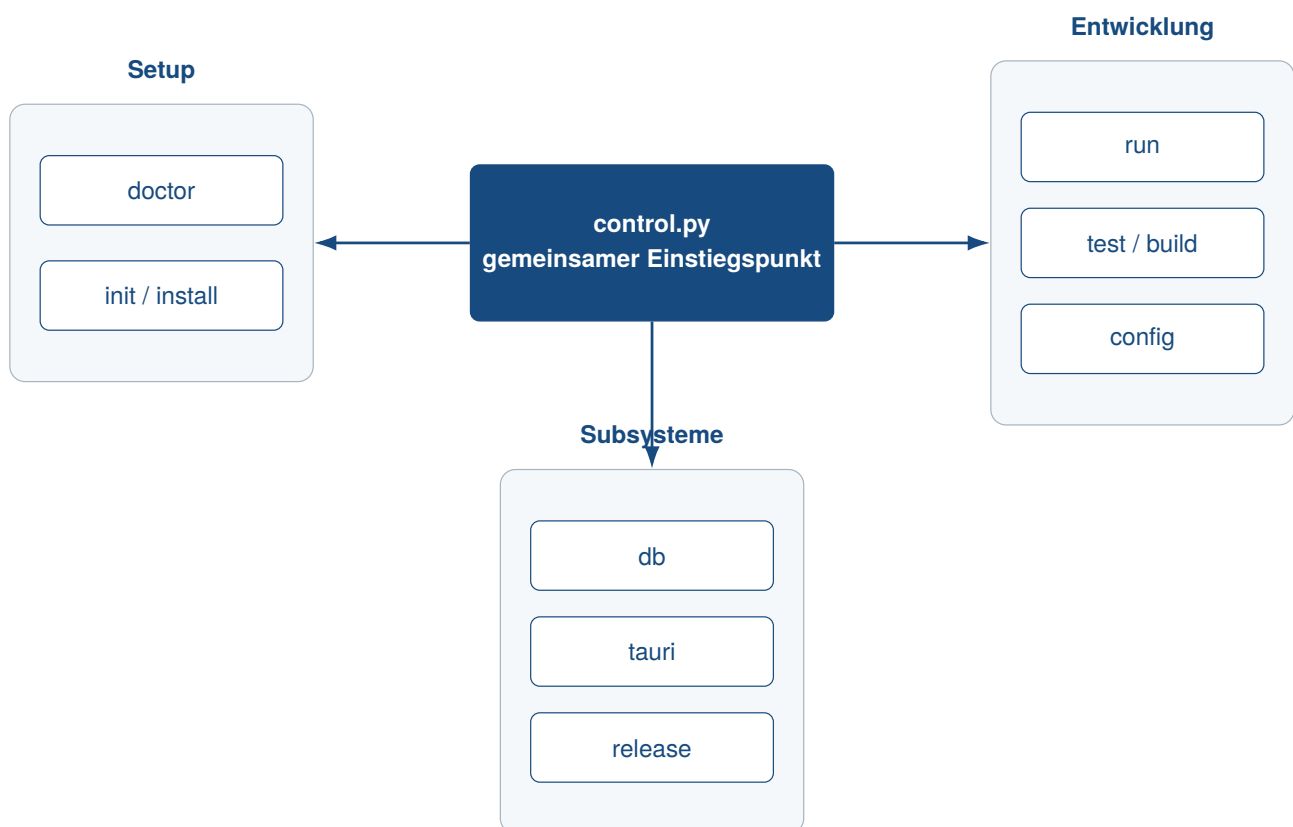


Abbildung 9: Befehlsgruppen des zentralen Projekt-Toolings

5.2 Projektinitialisierung und Generierung

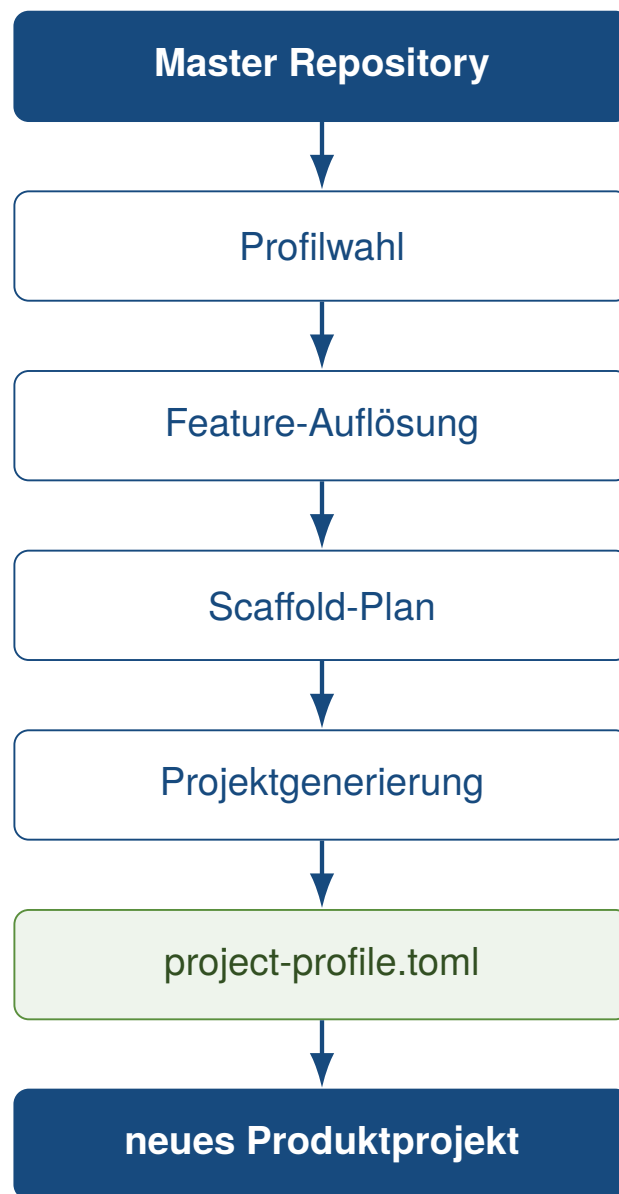


Abbildung 10: Workflow der profilgesteuerten Projektgenerierung

5.3 Systemprüfung und Installation

5.4 Entwicklungsbetrieb



Abbildung 11: Grundstruktur des Entwicklungsworkflows

5.5 Tests und Builds

5.6 Release- und Packaging-Workflow

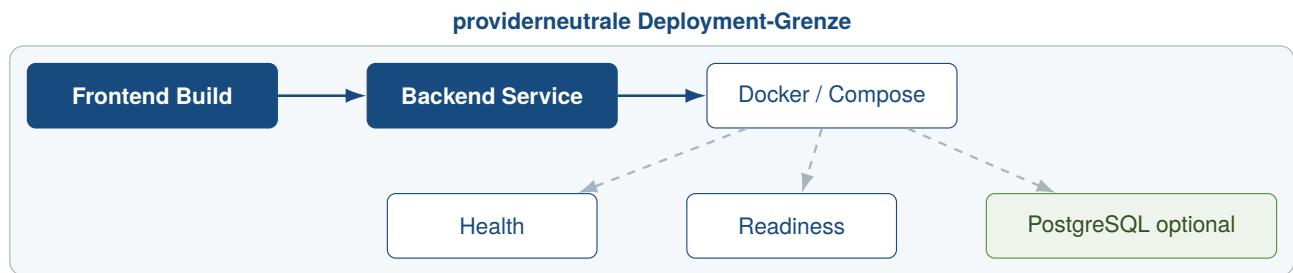


Abbildung 12: Providerneutrale Deployment-Architektur

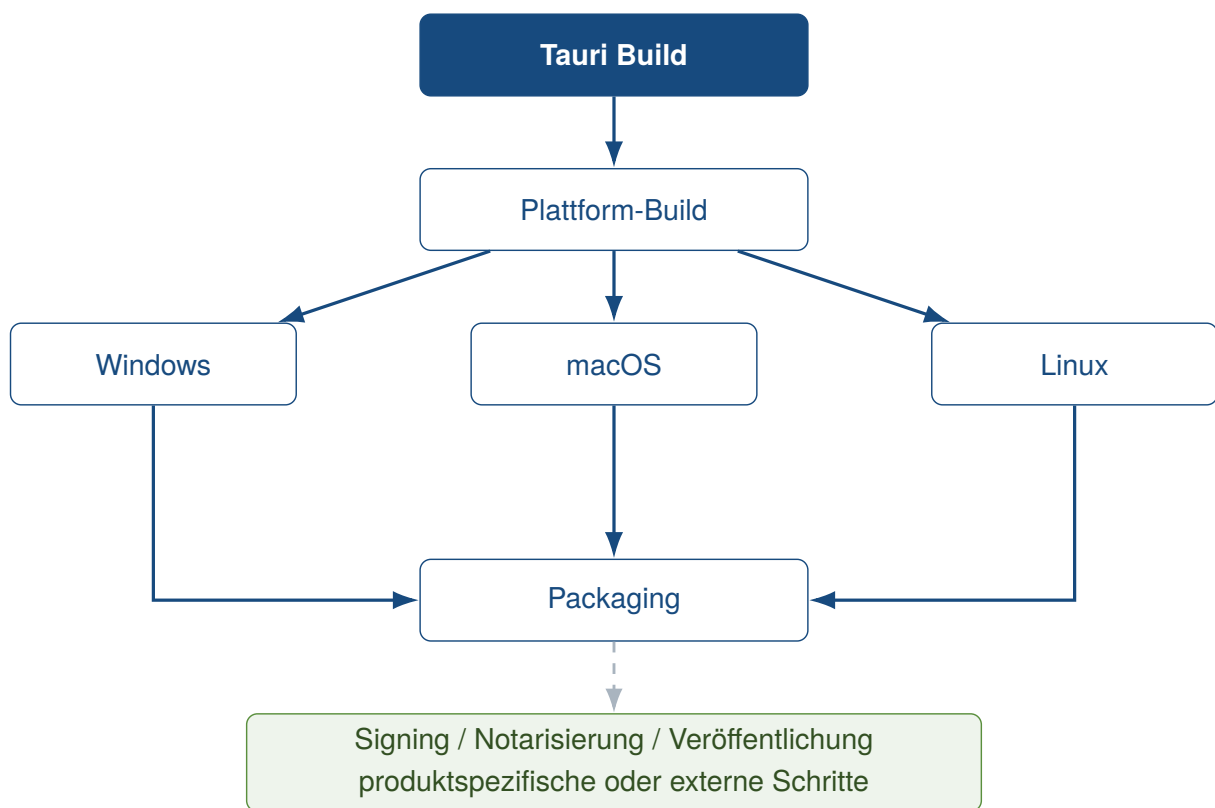


Abbildung 13: Plattformübergreifendes Modell für Desktop-Releases

5.7 Entwickler-Onboarding

6 Qualitätssicherung und Verifikation

6.1 Teststrategie



Abbildung 14: Nachweiskette der technischen Verifikation

6.2 Continuous Integration

6.3 Abnahme und technische Verifikation

Profil	Generate	Install	Doctor	Tests	Web Build	Desktop / Tauri	Database	Container
web-only	offen	offen	offen	offen	offen	offen	offen	offen
web-cloud	offen	offen	offen	offen	offen	offen	offen	offen
desktop-local	offen	offen	offen	offen	offen	offen	offen	offen
desktop-cloud	offen	offen	offen	offen	offen	offen	offen	offen
full-platform	offen	offen	offen	offen	offen	offen	offen	offen

Abbildung 15: Struktur der profilbezogenen Verifikationsmatrix

Tabelle 3: Struktur zur Dokumentation der technischen Verifikation

Profil	Prüfbereich	Kriterium	Evidenz	Ergebnis
--------	-------------	-----------	---------	----------

6.4 Reproduzierbarkeit

6.5 Security und Konfigurationsgrenzen

6.6 Extern verbleibende Prüfungen

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

7.2 Stärken

Tabelle 4: Struktur zur Gegenüberstellung von Stärken und Grenzen

Aspekt	Stärke / Evidenz	Grenze / Evidenz	Auswirkung
--------	------------------	------------------	------------

7.3 Schwächen und Grenzen

7.4 Wartungsaufwand

7.5 Vergleich mit alternativen Template-Strategien

7.6 Gesamtbewertung

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Projekttyp	Anforderungsfit	Anpassungsbedarf	Evidenz
Web-Anwendungen	offen	offen	offen
Cloud-Anwendungen	offen	offen	offen
Desktop-Anwendungen	offen	offen	offen
SaaS + Desktop	offen	offen	offen
lokale Business-Tools	offen	offen	offen
stark native Anwendungen	offen	offen	offen
Echtzeitsysteme	offen	offen	offen
Spiele	offen	offen	offen

Abbildung 16: Offene Bewertungsstruktur für die Projekteignung

Tabelle 5: Struktur zur Bewertung der Projekteignung

Projekttyp	Anforderungsfit	Anpassungsbedarf	Grenzen	Evidenz
------------	-----------------	------------------	---------	---------

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

8.3 Analyse von Kundenanforderungen

8.4 Auswahl des Projektprofils

8.5 Generierung eines Kundenprojekts



Abbildung 17: Prozess vom Kundenbedarf bis zur Auslieferung

8.6 Überführung in die Produktentwicklung

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

9.2 Beantwortung der Fragestellungen

9.3 Kritische Würdigung

9.4 Ausblick

9.5 Fazit

Literaturverzeichnis